

Resumen — Clase TypeScript (tipos, objetos, interfaces y clases)

Temas trabajados en clase, explicados desde cero, según los archivos de esta carpeta.

Archivo	Tema
<code>ejecutar_ts.md</code>	Cómo ejecutar TypeScript (npm/npx, tsx, ts-node, tsc)
<code>index.ts</code>	Tipos de datos, arrays, any, uniones, matrices
<code>objetos.ts</code>	Tipar objetos: inline, type, interface, tipar funciones
<code>Planeta.ts</code>	Clases, atributos, constructor, instancias

0. Antes de arrancar: ¿por qué TypeScript?

JavaScript es **flexible**: una variable puede empezar siendo un número y terminar siendo un texto, y nadie te avisa. Eso está bien para cosas chicas, pero en el backend esa libertad se paga cara: el error no aparece cuando lo escribís, aparece **cuando el programa ya está corriendo** y un usuario lo rompe.

TypeScript agrega **tipos**: le decís de antemano qué guarda cada cosa, y el editor te marca el error **mientras escribís**, antes de ejecutar nada.

 **Analogía:** JavaScript es escribir un mensaje y mandarlo. TypeScript es escribirlo con el corrector activado: te subraya el error antes de que lo lea nadie.

Dato clave que explica todo lo demás: ni el navegador ni Node entienden TypeScript. TS existe **solo mientras programás**. Antes de ejecutarse hay que convertirlo a JavaScript. Por eso la primera pregunta de la clase fue: *¿cómo ejecuto un archivo .ts?*

1. Ejecutar TypeScript

npm vs npx — la confusión clásica

Son dos comandos distintos y se usan en momentos distintos:

- **npm** = *node package manager*. Se usa para **instalar** paquetes.
- **npx** = se usa para **ejecutar** un paquete que ya tenés instalado; accede a los paquetes instalados globalmente.

💡 Regla mental: **npm instala, npx ejecuta**.

Las tres formas de ejecutar un `.ts`

Forma	Instalación	Ejecución	Cuándo usarla
tsx ★	<code>npm i --global tsx</code>	<code>npx tsx index.ts</code>	Desarrollo . La más directa y rápida
ts-node	<code>npm i --global ts-node</code>	<code>ts-node index.ts</code>	Clásica, pero más lenta y necesita un proyecto de Node
tsc	<code>npm i --global tsc</code>	<code>tsc index.ts → node index.js</code>	Antes de producción

¿Por qué **tsc** es distinta? **tsc** no ejecuta nada: **transpila**, o sea traduce. Hace **dos pasos**:

1. Lee tu `index.ts` y **crea un `index.js`** (JavaScript puro, ya sin tipos).
2. Recién ahí vos ejecutás ese `.js` con `node index.js`.

Las otras dos (**tsx**, **ts-node**) hacen ese mismo trabajo pero **en memoria y de una sola vez**, sin dejarte el archivo `.js` dando vueltas. Por eso son cómodas para desarrollar.

💡 En clase usamos **tsx** porque es la más rápida y no ensucia la carpeta con archivos `.js` generados.

⚠️ **Ojo:** el `.js` que genera **tsc** **no tiene tipos**. Los tipos desaparecen al transpilar. Sirven para protegerte a vos mientras escribís, no en tiempo de ejecución.

2. Tipos de datos y arrays (`index.ts`)

2.1 Tipos básicos

La sintaxis es simple: después del nombre, dos puntos y el tipo.

```
let edad: number = 33
```

```
let verdadero: boolean = false
```

```
let nombre: string = "javier"
```

A partir de acá, si intentás `edad = "treinta y tres"`, TypeScript te frena.

💡 TS también **infiere** el tipo solo: si escribís `let edad = 33`, ya sabe que es `number` aunque no lo aclares. Igual conviene tiparlo explícitamente cuando el valor no es obvio.

2.2 El tipo `any` — la puerta de escape (peligrosa)

`any` significa "cualquier cosa". Hay dos maneras de terminar con un `any`:

// 1) SIN QUERER: declarás sin tipo y sin valor inicial

```
let valor // TS no sabe qué es → any implícito
```

```
valor = "hola" // ✅ lo acepta
```

```
valor = 2 // ✅ también
```

// 2) A PROPÓSITO: lo forzás → tipado dinámico

```
let valorDinamico: any
```

```
valorDinamico = true
```

```
valorDinamico = 10
```

```
valorDinamico = ""
```

```
valorDinamico = undefined
```

```
valorDinamico = null
```

⚠️ **No hagas esto:** usar `any` para "que deje de molestar el error". Cuando ponés `any`, **apagás TypeScript** en esa variable: volvés al tipado dinámico de JavaScript y perdés toda la protección y el autocompletado. Si no sabés qué tipo poner, casi siempre existe una opción mejor (una unión, una `interface`).

2.3 Uniones — "puede ser esto o esto"

Cuando una variable legítimamente puede tener más de un tipo, se usa la barra `|`:

```
let valorMixto: (string | null)
```

Esto dice: puede ser un texto **o** puede estar vacío (`null`), **pero nada más**. Es la diferencia con `any`: `any` acepta todo; la unión acepta **solo lo que vos permitís**.

💡 Este es el caso típico de "todavía no cargó" o "no encontré el dato": `string | null`.

2.4 Arrays

Tipado común (el que más vas a usar):

```
let nombres: string[] = ["juan", "javier", "agustin"] // array de strings
```

```
let numeros: number[] = [1, 23, 4, 7] // array de numbers
```

Se lee así: `string [ ]` = "array de strings".

Tipado genérico (otra forma de escribir lo mismo):

```
let frutas2: Array<string>
```

`Array<string>` y `string [ ]` son **exactamente equivalentes**. La segunda es más corta y más común.

¿Y si quiero guardar más de un tipo en un array? Se combina unión + array:

```
let mixto: (string | number | boolean)[] = [1, 2, 3, "sty", false, true, "asd"]
```

⚠ Fijate en los **paréntesis**: `(string | number) [ ]` es "array de strings o numbers". Sin paréntesis, `string | number [ ]` significa otra cosa distinta: "un string, o un array de numbers".

2.5 Matrices — arrays de arrays

Una matriz es simplemente **un array cuyos elementos son arrays**. Por eso el tipo lleva dos corchetes: `number [ ] [ ]`.

```
let matriz: number[][] = [
```

```
    [1,2,3], // Fila 0
```

```
    [4,5,6], // Fila 1
```

```
    [7,8,9] // Fila 2
```

```
]
```

```
console.log(matriz[1][1]) // 5
```

Cómo se lee `matriz[1][1]`: primero elegís la **fila**, después la **columna**.

- `matriz[1]` → agarra la fila 1 → `[4, 5, 6]`
- `[1]` de eso → agarra el elemento 1 de esa fila → 5

💡 Recordá que se cuenta **desde 0**: la primera fila es la 0, no la 1.

3. Tipar objetos (`objetos.ts`)

Un objeto agrupa varios datos relacionados (un usuario, un producto). La pregunta de esta sección es: **¿cómo le describo a TypeScript la "forma" que debe tener ese objeto?**

Vimos tres caminos, de peor a mejor.

A. Inline / directa — el primer intento

Se escribe el tipo pegado a la variable:

```
let persona1: {  
  
  nombre: string;  
  
  id: number;  
  
  edad: number;  
  
  activo: boolean;  
  
} = {  
  
  nombre: "valentino", id: 1, edad: 16, activo: true  
  
}
```

-  **Pro:** fácil, no hay que aprender nada nuevo.
-  **Contra:** no es práctico, hay que repetir código. Si mañana necesitás 10 personas, copiás esa descripción 10 veces. Y si agregás un campo, lo tenés que cambiar en los 10 lugares.

Ese problema es exactamente lo que resuelven `type` e `interface`.

B. type — armar nuestro propio molde

Es **la forma más simple que existe** de crear un tipo propio. Definís el molde **una vez** y lo reutilizás.

```
type Persona = {  
  
  nombre: string;  
  
  id: number;  
  
  edad: number;  
  
  activo: boolean;  
  
}
```

// Instancia -> es lo que sale del molde

```
let felix: Persona = { nombre: "felix", id: 2, edad: 23, activo: true }
```

 **Molde e instancia:** Persona es el molde (la forma). felix es **lo que sale del molde**. El molde no es una persona: es la descripción de cómo debe ser una.

type no es solo para objetos. También sirve para ponerle nombre a una unión, y ahí se vuelve muy útil:

```
type ID = string | number
```

```
type estadoDeCarga = "cargando" | "descargado" | "cargado"
```

El segundo caso es un **tipo literal**: estadoDeCarga no acepta cualquier texto, acepta **exactamente** una de esas tres palabras. Si escribís "cargandoo" con typo, TS te lo marca.

 Este patrón es oro para estados: evita errores de tipeo que en JS pasarían desapercibidos.

C. interface — el contrato

Es un **contrato o acuerdo** de cómo algo debe ser, y se debe cumplir.
Garantiza que se cumplan características.

Sus reglas, tal como las vimos:

- Puede **definir métodos, pero sin implementarlos** (dice *qué* debe existir, no *cómo* funciona).
- **No se puede instanciar**, solo se usa para tipar.
- Es **solo para objetos** (a diferencia de type, que también hace uniones).

```
interface Pizza {

    salsa?: boolean,    // ? = OPCIONAL, puede no estar

    masa: boolean,      // sin ? = OBLIGATORIA

    queso?: boolean,

    condimentar(condimento: string): string // método: firma sin implementación

}
```

Cuando creás un objeto de ese tipo, **estás obligado a cumplir el contrato**: poner masa e implementar condimentar.

```
let pizzaCancha: Pizza = {

    salsa: true,

    masa: true,

    queso: false,

    condimentar: function(condimento: string) {

        return `Ponele ${condimento}` // acá SÍ se implementa

    }

}
```

El **?** (**propiedad opcional**) es importante: `salsa?` significa que el objeto puede tener salsa o no, y en ambos casos cumple el contrato. Sin el `?`, si falta, TS da error.

Y una interface también tipa arrays, porque es un tipo como cualquier otro:

```
let pizzas: Pizza[] = [ /* varias pizzas, todas cumpliendo el contrato */ ]
```

 **Error común:** olvidarte de implementar el método del contrato. `let p: Pizza = { masa: true } → ❌ Property 'condimentar' is missing. Se arregla` agregando la propiedad que falta: el contrato no es opcional.

D. Tipar funciones

En una función se tipan **dos cosas**: lo que entra (parámetros) y lo que sale (retorno). El tipo de retorno va después de los paréntesis.

```
function saludar(persona: Persona): string {  
  
    if (persona.activo) {  
  
        return `Hola ${persona.nombre} estas activo!!`  
  
    } else {  
  
        return `Hola ${persona.nombre} estas inactivo!!`  
  
    }  
  
}
```

Se lee: "recibe una Persona y **devuelve un string**".

Retorno con unión — cuando la función puede devolver dos cosas distintas:

```
function pizzaFavorita(persona: Persona, pizza: Pizza): string | null {  
  
    if (persona.activo) {  
  
        return pizza.condimentar("oregano") // devuelve string  
  
    } else {  
  
        return null // devuelve null  
  
    }  
  
}
```

El : string | null es **honesto**: avisa a quien use la función que puede recibir null y que conviene chequearlo antes de usarlo.

💡 Tipar el retorno es lo que convierte a la función en un **contrato**: quien la llama sabe exactamente qué esperar sin leer el código de adentro.

4. Clases (Planeta.ts)

Interface vs Clase — la diferencia clave

- **Interface** → **contrato**. Dice qué debe existir. No se instancia, no implementa.
- **Clase** → **molde y la fábrica**. Además de describir, **puede contener la implementación de los métodos** y fabrica objetos.

O sea: la interface **describe**, la clase **describe y produce**.

Anatomía de una clase

```
class Planeta {  
  
    // 1) Atributos / Propiedades -> características  
  
    nombre: string;  
  
    masaKg: number;  
  
    radioKm: number;  
  
    tieneAtmosfera: boolean;  
  
    tipo: string;  
  
    distanciaEstrella: number;  
  
    // 2) Constructor -> máquina de ensamble  
  
    constructor(nombre: string, masaKg: number, radioKm: number,  
        tieneAtmosfera: boolean, tipo: string, distanciaEstrella: number) {  
  
        this.nombre = nombre;  
  
        this.masaKg = masaKg;  
  
        // ... y así con cada propiedad  
    }  
  
    // 3) Métodos (lo que el objeto sabe hacer)  
  
}
```

Las tres partes:

1. **Atributos/Propiedades** = las **características** del objeto (qué sabe).
2. **Constructor** = la **máquina de ensamble**: toma los elementos que necesita y **arma la instancia**. Se ejecuta automáticamente al hacer new. Sus parámetros son exactamente los que vas a pasar en new Planeta(...).
3. **Métodos** = lo que el objeto sabe hacer.

El this

Dentro del constructor aparece `this.nombre = nombre`. Se lee: *"la propiedad nombre de este objeto que estoy armando vale el nombre que me pasaron por parámetro"*.

- `this.nombre` → la propiedad de la instancia.
- `nombre` (sin `this`) → el parámetro que llegó al constructor.

 **Error común:** olvidar el `this` y escribir `nombre = nombre`. No falla visiblemente, pero **la propiedad queda vacía**, porque estás asignando el parámetro a sí mismo en vez de guardarlo en el objeto.

Instanciar — usar la fábrica

```
let saturno = new Planeta("saturno", 200, 5.68E26, true, "planeta", 8000000)
```

```
let pluton = new Planeta("pluton", 200, 5.68E26, true, "planeta", 8000000)
```

`new` dispara el constructor y devuelve un objeto nuevo. `saturno` y `pluton` son **dos instancias independientes** del mismo molde: cambiar una no afecta a la otra.

 El **orden de los argumentos importa** y debe coincidir con el del constructor. Si intercambiás dos parámetros del mismo tipo, TS no puede avisarte — el error es lógico, no de tipos.

Conceptos clave para recordar

Distinción	En una línea
Molde vs instancia	El molde (<code>type/class</code>) describe; la instancia es lo que sale del molde
type vs interface	<code>type</code> sirve para objetos y uniones; <code>interface</code> es solo para objetos, expresa un contrato y puede declarar métodos
interface vs class	La interface no se instancia y no implementa; la clase sí (es la fábrica)

Distinción	En una línea
any vs unión	any acepta todo y apaga TS; la unión acepta solo lo que vos permitís
npm vs npx	npm instala, npx ejecuta
tsc vs tsx	tsc genera el .js (dos pasos); tsx transpila y ejecuta de una

Checklist "ya puedo..."

- ☐ Ejecutar un .ts con npx tsx archivo.ts y explicar por qué hace falta.
- ☐ Tipar variables, arrays, arrays mixtos y matrices.
- ☐ Explicar por qué any es peligroso y qué usar en su lugar.
- ☐ Crear un molde con type y una unión de literales.
- ☐ Escribir una interface con propiedades opcionales y un método.
- ☐ Tipar los parámetros y el retorno de una función.
- ☐ Crear una clase con constructor e instanciarla con new.